

PROGRAMAREA CALCULATOARELOR ȘI LIMBAJE DE PROGRAMARE I

Tema #1 Funcții, Vectori și Matrici

Deadline soft: 14.11.2025 23:55. Deadline hard: 17.11.2025 23:55

Coordonator: Radu Nichita

Responsabili temă: Alexandru-Mihail-Iulian Buzea, Victor Sava, Mihnea Grigore

Changelog:

- 01.11.2025: Publicare tema 1.
Vă rugăm să adresați toate întrebările legate de teme pe forumul asociat temei 1 de pe site-ul de curs. Orice clarificare / corectare / actualizare legată de enunț, checker sau lucruri administrative etc, vor fi anunțate exclusiv pe forumul dedicat temei 1.
- 01.11.2025: Publicare checker.
- 02.11.2025: Publicare checker online (*vmchecker-next*). Checker-ul de pe Moodle (*vmchecker-next*) a fost publicat. Enunțul temei a rămas identic, excepție face intrarea curentă din changelog (adăugată pentru vizibilitate).

Obiectivele temei

După această temă, studenții vor rezolva în limbajul C o serie de probleme ce folosesc funcții, vectori și matrici. În urma acestui exercițiu, studenții vor putea să:

- să abordeze o problemă mai complexă și să o împartă în subprobleme mai mici ce pot fi rezolvate individual
- să scrie cod lizibil și ușor de menținut
- să utilizeze funcțiile din limbajul C de citire și scriere a datelor
- să utilizeze funcțiile matematice din C
- să utilizeze structuri condiționale și repetitive din C
- să utilizeze tablouri (vectori și matrici) în limbajul C
- să utilizeze tool-ul de automatizări Make pentru a compila automat programele scrise

Contents

Problema 1 - Window join	3
Problema 2 - Akari	6
Problema 3 - Helicopters	13
Regulament	18
Arhivă	18
Checker	18
Punctaj	18
Reguli și precizări	18
Alte precizări	19

Problema 1 - Window join

Enunț

Într-un templu vechi al numerelor și al timpului, un oracol veghează asupra unui flux nesfârșit de date. Cei care vor să-i descifreze misterul aud mai întâi versurile sale criptice:

În templul codului, curge un fir,
De numere-n timp, venind în sir.
Oracolul spune - cu glasul stins:
„Ascultă-mă bine, că tot am învins:
Când timpul din urmă nu trece de-un prag,
Adună-le-n taină, nu le lăsa vag.
Caută 'ntre ele legământul divin:
Cmmdc- ce le ține-n destin,
Și cmmmc - ce le face-mpreună,
Să bată la fel, sub aceeași lună.
Apoi, din fluxul ce curge-n tăcere,
Trimite la ieșire perechi de putere.”

Se dă un flux de numere întregi, fiecare însoțit de timestamp-ul (momentul de timp) la care a fost primit. De asemenea, se primește o valoare **window**, care reprezintă o durată de timp (în unități arbitrare). Pentru fiecare nou număr primit, se consideră toate numerele primite în ultimele **window** unități de timp (inclusiv cel curent). Se cere determinarea CMMDC-ului și a CMMMC-ului a tuturor perechilor din **window**.

Restricții și precizări

- Pentru prima pereche din flux, nu se va afișa nimic
- Numărul de valori ce va fi procesat va fi maxim 10^4
- Dimensiunea maximă a unui window va fi maxim 400
- Fiecare timestamp t_i este număr natural nenul pe maxim 64 biți
- Fiecare valoare x_i e un număr natural pe maxim 64 biți
- Timestamp-urile intrărilor (t_i, x_i) sunt unice, și intrările sunt date în ordinea strict crescătoare a acestor timestamp-uri.
- Cel mai mic multiplu comun a oricare două numere nu poate depăși $2^{64} - 1$ (o valoare pe 64 de biți)
- Rezultatele join-ului (cel mai mic multiplu comun, respectiv cel mai mare divizor comun) apar în ordinea strict crescătoare a timestamp-urilor (vor apărea sortate crescător după timestamp-ul intrării cu timestamp mai mic, apoi după celălalt timestamp în caz de egalitate)
- Fluxul de intrări se termină întotdeauna cu intrarea cu timestamp 0 și valoare 0, care nu va fi luată în calcul la formarea join-urilor

Scrieți un program în limbajul C care să primească ca input flux-ul de date și să returneze CMMMC și CMMDC pentru fiecare moment în care sosește un număr nou, conform precizărilor din enunț.

Date de intrare

Toate datele se vor citi de la **STDIN**.

- Pe prima linie se află un număr întreg **window**.
- Pe următoarele linii, perechi de forma **t x**, unde **t** este timestamp-ul și **x** este valoarea.
- Fluxul de intrări se termină cu linia **0 0**, care nu va fi procesată (nu se va afișa nimic pentru această linie)

Între oricare doi tokeni vizibili din input există exact un spațiu. La finalul oricărei linii din input există un caracter **newline**.

```
1 window
2 t1 x1
3 t2 x2
4 ...
5 tn xn
6 0 0
```

Date de ieșire

Toate datele se vor afișa la **STDOUT**.

- Pe fiecare linie se va afișa o pereche de numere **CMMMC CMMDC**, reprezentând cel mai mic multiplu comun și cel mai mare divizor comun pentru o pereche de numere validă din fereastra de timp.

Între oricare doi tokeni vizibili din output există exact un spațiu. La finalul oricărei linii din output există un caracter **newline**.

Exemplul 1

Date de intrare

```
1 5
2 1 6
3 2 8
4 7 9
5 9 3
6 0 0
```

Date de ieșire

```
1 24 2
2 72 1
3 9 3
```

Explicație 1

1. Primul join se realizează între evenimentele cu $t = 1$ și $t = 2$ (intervalul dintre ele este $2 - 1 = 1 \leq 5$), care au valorile 6 și 8. $\text{cmmmc}(6, 8) = 24$, $\text{cmmdc}(6, 8) = 2$ deci prima ieșire va fi perechea (24, 2).
2. Al doilea join se realizează între evenimentele cu $t = 2$ și $t = 7$ (intervalul dintre ele este $7 - 2 = 5 \leq 5$), care au valorile 8 și 9. $\text{cmmmc}(8, 9) = 72$, $\text{cmmdc}(8, 9) = 1$ deci a doua ieșire va fi perechea (72, 1).
3. Al treilea join se realizează între evenimentele cu $t = 7$ și $t = 9$ (intervalul dintre ele este $9 - 7 = 2 \leq 5$), care au valorile 9 și 3. $\text{cmmmc}(9, 3) = 9$, $\text{cmmdc}(9, 3) = 3$ deci a treia ieșire va fi perechea (9, 3).
4. Perechile din rezultat apar în ordinea dată pentru că au fost sortate după cel mai mic timestamp ale celor două intrări joined pentru a se obține ieșirea respectivă.

Exemplul 2**Date de intrare**

```
1 4
2 1 2
3 2 8
4 5 5
5 8 10
6 9 6
7 12 15
8 0 0
```

Date de ieșire

```
1 8 2
2 10 1
3 40 1
4 10 5
5 30 1
6 30 2
7 30 5
8 30 3
```

Explicație 2

Join-urile realizate sunt (în această ordine):

- de la $t = 1$ la $t = 2$
- de la $t = 1$ la $t = 5$
- de la $t = 2$ la $t = 5$
- de la $t = 5$ la $t = 8$
- de la $t = 5$ la $t = 9$
- de la $t = 8$ la $t = 9$
- de la $t = 8$ la $t = 12$
- de la $t = 9$ la $t = 12$

Problema 2 - Akari

Enunț

Akari, cunoscut și sub numele de **Light Up**, este un joc de logică care se desfășoară pe un grid dreptunghiular de dimensiuni $n \times m$ format din celule albe și negre. Scopul jocului este să plasezi becuri în celulele albe astfel încât fiecare celulă albă din grid să fie iluminată. Un bec luminează orizontal, vertical și pe el însuși pe grid. Lumina se proiectează până când drumul este blocat de o celulă neagră sau de marginea grid-ului.

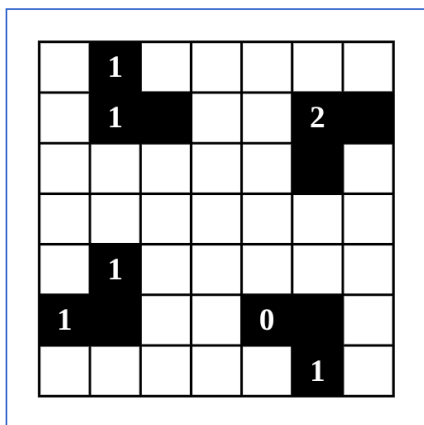
Celulele negre pot avea și numere de la **0 la 4**, indicând exact câte becuri trebuie plasate în celulele adiacente (sus, jos, stânga, dreapta) din jurul acelei celule negre. Pentru celulele negre fără numere nu există restricții privind numărul de becuri adiacente.

Astfel, putem defini câteva reguli de bază ale Akari:

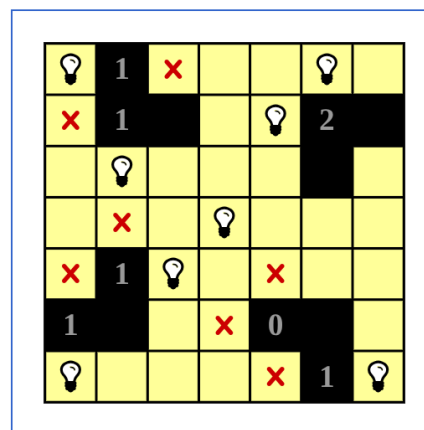
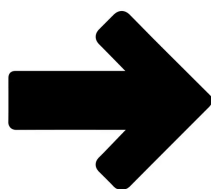
- **Becuri care se luminează reciproc:** Două becuri nu pot să se ilumineze reciproc (nu trebuie să existe un bec pe aceeași linie sau coloană cu alt bec fără o celulă neagră între ele);

- **Numărul corect de becuri adiacente pentru celulele negre numerotate:** Fiecare celulă neagră numerotată indică numărul exact de becuri ce trebuie plasate în celulele adiacente (sus, jos, stânga, dreapta). Pe parcursul jocului celulele numerotate pot avea un număr mai mic de becuri adiacente.

Pentru a câștiga jocul fiecare celulă albă trebuie să fie iluminată de cel puțin un bec astfel încât toate regulile Akari să fie respectate și grid-ul să fie complet iluminat. Grid-ul are o soluție și aceasta este unică.



Grid nerezolvat



Grid rezolvat

Pentru aceasta problemă există două cerințe posibile:

- **Opțiunea 1 - Completarea grid-ului neiluminat**

Pentru un grid cu câteva becuri deja plasate, celule negre și celule numerotate, umpleți restul grilei cu marcaje 'x', care indică celule iluminate unde nu pot fi plasate becuri, astfel încât sunt respectate regulile Akari.

- **Opțiunea 2 - Validarea grid-ului**

Verificați pentru un grid dat dacă respectă regulile Akari.

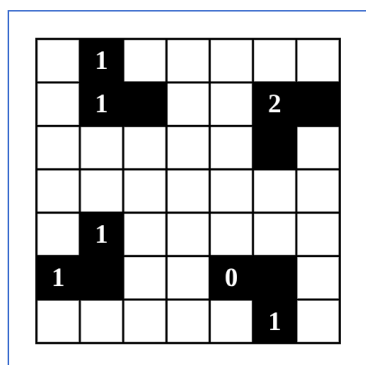
Hint: Sugerăm rezolvarea manuală a unui grid pentru înțelegerea problemei. Recomandăm următoarea platformă pentru rezolvarea grid-urilor: [Light Up](#)

Codificare

Pentru a transforma grid-ul grafic în simboluri, vom folosi următoarele convenții:

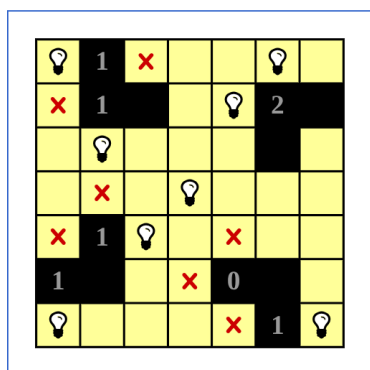
- Becurile (Light) se vor codifica folosind litera 'L';
- Celulele negre fără număr se vor codifica folosind caracterul '#';
- Celulele negre vor avea numărul efectiv pe celulă;
- Celulele albe se vor codifica folosind caracterul '-';
- Celulele galbene/iluminate și celule unde nu se pot pune becuri se vor codifica folosind caracterul 'x'.

Pentru grid-urile de mai sus avem codificarea dată:



Grid nerezolvat

1	-	1	-	-	-	-
2	-	1	#	-	-	2 #
3	-	-	-	-	-	# -
4	-	-	-	-	-	-
5	-	1	-	-	-	-
6	1	#	-	-	0	# -
7	-	-	-	-	-	1 -



Grid nerezolvat

1	L	1	x	x	x	L x
2	x	1	#	x	L	2 #
3	x	x	x	x	x	# x
4	x	x	x	x	x	x x
5	x	1	x	x	x	x x
6	1	#	x	x	0	# x
7	x	x	x	x	x	1 x

Restricții și precizări

- $7 \leq m, n \leq 50$
- Tabloul bidimensional este indexat de la 0
- Becul reprezintă o celulă iluminată
- Pentru opțiunea 1, se garantează ca grid-ul este valid

Date de intrare

Toate datele se vor citi de la **STDIN**.

- Pe prima linie se află un număr întreg **o** (1 sau 2), reprezentând opțiunea.
- Pe următoarea linie se află două numere întregi **n** și **m**, reprezentând numărul de linii și, respectiv, numărul de coloane ale grid-ului.
- Pe următoarele **n** linii se află câte **m** caractere (separate prin spații), reprezentând grid-ul.

```
1 o
2 n m
3 grid[0][0] grid[0][1] ... grid[0][m-1]
4 grid[1][0] grid[1][1] ... grid[1][m-1]
5 ...
6 grid[n-1][0] grid[n-1][1] ... grid[n-1][m-1]
```

Date de ieșire

Toate datele se vor afișa la **STDOUT**.

- Pentru **Opțiunea 1**: Se va afișa grid-ul modificat (o matrice de **n** linii și **m** coloane de caractere, separate prin spații).
- Pentru **Opțiunea 2**: Se va afișa un singur cuvânt: "ichi" dacă grid-ul respectă regulile, sau "zero" în caz contrar.

Exemplul 1**Date de intrare**

```

1 1
2 7 7
3 - - - - # 2 -
4 1 - 0 - - - -
5 0 - - - - # -
6 - - - - - -
7 - 1 - - - L 1
8 - - L - # - #
9 - # 2 - - - -

```

Date de ieșire

```

1 - - x - # 2 -
2 1 x 0 x - - -
3 0 x x - - # -
4 x - x - - x x
5 - 1 x x x L 1
6 x x L x # x #
7 - # 2 - - x -

```

Explicație 1

Opțiunea este o == 1, deci vom rezolva cerința 1. Programul procesează fiecare bec existent pe grid.

Pentru fiecare bec ('L'):

- Propagă lumina orizontal și vertical de la poziția becului.
- Lumina se extinde până întâlnește o celulă neagră ('#', '0'-'4') sau marginea grid-ului.
- Toate celulele albe ('-') traversate de lumină devin celule iluminate ('x').

După propagarea luminii de la ambele becuri ('L' la (4,5) și 'L' la (5,2)), grid-ul va fi actualizat astfel încât toate celulele albe iluminate devin marcate cu 'x'.

Pentru celulele negre numerotate:

- Dacă numărul de becuri adiacente este egal cu numărul de pe celula numerotată atunci celulele adiacente se marchează cu 'x' întrucât nu mai pot fi plasate becuri ('L'), iar celelalte celule adiacente (becuri, negre) rămân neschimbate.
- Dacă numărul de becuri adiacente este mai mic decât numărul de pe celula numerotată atunci celulele adiacente rămân neschimbate.

Pentru exemplul dat celulele adiacente cu celulele '0' ((1,2) și (2,0)) sunt marcate cu 'x', iar celula '1' la poziția (4,6) are exact 1 bec adiacent, deci restul celulelor adiacente albe sunt marcate cu 'x'.

Exemplul 2**Date de intrare**

```

1 2
2 7 7
3 - - - - # 2 -
4 1 - 0 - - - -
5 0 - - - - # -
6 - - - - - -
7 - 1 - - - L 1
8 - - L - # - #
9 - # 2 - - - -

```

Date de ieșire

```

1 ichi

```

Explicație 2

Opțiunea `o == 2`, deci vom rezolva cerința 2.

Programul verifică dacă grid-ul, în starea sa actuală, respectă regulile Akari:

- **Becuri care se luminează reciproc:** Se verifică dacă există două becuri pe aceeași linie sau coloană, fără o celulă neagră între ele. În acest grid becurile sunt izolate, deci această regulă este respectată.
- **Numărul corect de becuri adiacente pentru celulele negre numerotate:** Se parcurg toate celulele negre care au un număr ('0'-'4') și se numără becurile din celulele adiacente.
 - De exemplu, celula neagră '2' de la (0,5) nu are becuri adiacente, deși necesită două, dar întrucât jocul nu este finalizat, celulele numerotate pot avea un număr mai mic de becuri învecinate. Același lucru se poate spune și despre celulele '1' de la (1,0) și (5,1) și celula '2' de la (6,2). Deci se respectă regula.
 - Celulele '0' de la (1,2) și (2,0) nu au becuri adiacente, deci se respectă regula.
 - Celula '1' de la (4,6) are exact numărul de becuri adiacente, deci se respectă regula.

Astfel, după parcurgerea tuturor celulelor negre numerotate, regula numărului corect de becuri adiacente pentru celulele negre numerotate se respectă. Cele 2 reguli Akari sunt respectate deci se afișează 'ichi'.

Exemplul 3**Date de intrare**

```

1 2
2 7 7
3 - - - - # 2 -
4 1 L 0 - - - -
5 0 - - - - # -
6 - - - - - L
7 - 1 L - - L 1
8 - - L - # - #
9 - # 2 - - - -

```

Date de ieșire

```

1 zero

```

Explicație 3

Opțiunea `o == 2`, deci vom rezolva cerința 2. Programul verifică dacă grid-ul, în starea sa actuală, respectă regulile Akari:

- **Becuri care se luminează reciproc:** Se verifică dacă există două becuri pe aceeași linie sau coloană, fără o celulă neagră între ele.
 - Becurile ('L') de la (4,2) și (5,2) sunt pe aceeași coloană deci regula NU se respectă.
 - La fel, regula NU se respectă pentru becurile de la (4,2) și (4,5), fiind pe aceeași linie.
- **Numărul corect de becuri adiacente pentru celulele negre numerotate:** Se parcurg toate celulele negre care au un număr ('0'-'4') și se numără becurile din celulele adiacente. Se face aceeași verificare ca la exemplul precedent, dar se observă că:
 - Celula '0' de la (1,2) are 1 bec adiacent, deci NU se respectă regula.
 - Celula '1' de la (4,6) are 2 becuri adiacente, deci NU se respectă regula.
 - Lumina propagată de becul de la poziția (3,6) ar bloca celula numerotată de la poziția (0,5) din a putea avea 2 becuri, astfel, regula NU se respectă.

Astfel, după parcurgerea tuturor celulelor negre numerotate, regulile Akari NU sunt respectate deci se afișează 'zero'.

Exemplul 4**Date de intrare**

```

1 2
2 7 7
3 - - - - 2 - -
4 - - - - # - -
5 # 2 - - - - -
6 - - - - - L -
7 - - - - - 2 1
8 - - 1 - - - -
9 - - # - - - -

```

Date de ieșire

```

1 zero

```

Explicație 4

Opțiunea `o == 2`, deci vom rezolva cerința 2. Programul verifică dacă grid-ul, în starea sa actuală, respectă regulile Akari:

- **Becuri care se luminează reciproc:** Regula se respectă.
- **Numărul corect de becuri adiacente pentru celulele negre numerotate:**
 - Lumina propagată de becul de la poziția (3,5) ar bloca celula numerotată de la poziția (0,4) din a putea avea 2 becuri, astfel, regula NU se respectă.

Regulile Akari NU sunt respectate deci se afișează 'zero'.

Problema 3 - Helicopters

Enunț

Un teren de fotbal este folosit pentru aterizarea a k elicoptere. Gazonul de pe stadion este parcat în **pătrățele de aceeași dimensiune** 1×1 , cu laturile paralele cu marginile terenului. Liniile cu pătrățele de gazon sunt numerotate de sus în jos cu numerele $1, 2, \dots, n$, iar coloanele cu pătrățele de gazon sunt numerotate de la stânga la dreapta cu numerele $1, 2, \dots, m$.

Din cauza tipului diferit de iarbă, se știe care dintre pătrățelele de gazon sunt afectate sau nu de umbra elicopterului. Acest lucru este precizat printr-un **tablou bidimensional** a cu n linii și m coloane, cu elemente 0 și 1 ($a_{ij} = 0$ înseamnă că pătrățul aflat pe linia i și coloana j este afectat de umbră, iar $a_{ij} = 1$ înseamnă că pătrățul aflat pe linia i și coloana j nu este afectat de umbră).

Fiecare elicopter are 3 roți pe care se sprijină. Roțile fiecărui elicopter determină un **triunghi dreptunghic isoscel**. Elicopterele aterizează astfel încât triunghiurile să aibă **catetele paralele cu marginile terenului**. Pentru a preciza poziția unui elicopter pe teren este suficient să cunoaștem **coordonatele (linia și coloana) capetelor ipotenuzei și poziția vârfului**: deasupra (codificată prin 1) sau dedesubtul ipotenuzei (codificată prin -1).

De asemenea, **ipotenuza trebuie să fie paralelă cu una dintre pseudodiagonalele matricei** (un segment oblic). O ipotenuză nu poate fi reprezentată de un segment vertical sau orizontal. În cazul în care un astfel de triunghi este întâlnit, se va afișa un **mesaj de eroare**: "Elicopterul $\langle indice \rangle$ este poziționat necorespunzător!" (**Atenție**: Mesajul nu va conține diacritice).

Un elicopter se consideră că **a aterizat greșit** dacă triunghiul format sub el (definit mai sus) are **mai mult de jumătate din pătrățele afectate de umbră**. Administratorul terenului de fotbal dorește să determine numărul X de elicoptere care nu afectează niciun pătrățel din teren și numerele de ordine ale elicopterelor care au aterizat greșit, în ordine crescătoare: $e_1, e_2, \dots, e_Y$, știind că există k elicoptere codificate prin numerele $1, 2, \dots, k$.

Administratorul terenului vă cere să îl ajutați pentru a verifica starea terenului de fotbal. Scrieți un program în **limbajul C** care să determine pentru datele din enunț: numărul de elicoptere X care nu afectează niciun pătrățel din teren și numerele de ordine ale elicopterelor care au aterizat greșit, în ordine crescătoare, precedate de numărul lor Y .

Restricții și precizări

- $2 \leq n, m \leq 1000$
- $0 \leq k \leq 800$
- Nu există suprapuneri de triunghiuri corespondente la elicoptere distincte
- Triunghiurile asociate elicopterelor valide conțin cel puțin trei pătrățele (nu există triunghiuri dreptunghice isoscele degenerate)

Date de intrare

Toate datele se vor citi de la **STDIN**.

- Pe prima linie se află două numere întregi n și m , cu semnificația din enunț
- Pe următoarele n linii se află câte m numere, cu valori de 0 sau 1 , reprezentând matricea a
- Pe următoarea linie se află numărul de elicoptere k
- Pe următoarele k linii se află câte 5 valori pentru fiecare elicopter: r_1, c_1, r_2, c_2, s (coordonatele vârfurilor ipotenuzei și $s = 1$ pentru vârf „deasupra”, -1 pentru „dedesubt”).

Între oricare doi tokeni vizibili din input există exact un spațiu. La finalul oricărei linii din input există un caracter **newline**.

```

1 n m
2 a[0][0] a[0][1] ... a[0][m-1]
3 ...
4 a[n-1][0] a[n-1][1] ... a[n-1][m-1]
5 k
6 r1_1 c1_1 r2_1 c2_1 s_1
7 r1_2 c1_2 r2_2 c2_2 s_2
8 ...
9 r1_k c1_k r2_k c2_k s_k

```

Date de ieșire

Toate datele se vor afișa la **STDOUT**.

- În cazul în care există W elicoptere invalide: pe primele W rânduri se vor afișa mesajele pentru triumphiurile necorespunzătoare (Exemplu: "Elicopterul $\langle indice \rangle$ este poziționat necorespunzător!").
- Pe rândul $W + 1$, se va afișa un număr întreg X , reprezentând numărul de elicoptere valide care nu afectează niciun pătrățel din teren.
- Pe rândul $W + 2$, se va afișa un număr întreg Y , reprezentând numărul de elicoptere valide care au aterizat greșit.
- Pe rândul $W + 3$, se vor afișa Y numere întregi, separate prin câte un spațiu, reprezentând numerele de ordine ale elicopterelor valide care au aterizat greșit, în ordine crescătoare.

La finalul oricărei linii din output există un caracter **newline**.

```

1 Mesaj invalid (1)
2 ...
3 Mesaj invalid (W)
4 X
5 Y
6 idx_1 idx_2 ... idx_Y

```

Exemplul 1**Date de intrare**

```

1 7 9
2 1 1 1 1 1 1 1 1 1
3 0 0 0 0 1 1 1 1 0
4 0 0 1 0 1 1 1 0 0
5 1 1 1 0 1 1 0 1 1
6 0 0 1 1 1 1 0 1 1
7 1 1 1 1 1 1 0 1 1
8 1 1 1 1 1 1 0 0 1
9 6
10 1 1 3 3 -1
11 1 9 5 5 1
12 5 1 6 2 1
13 7 3 7 5 -1
14 2 4 5 4 -1
15 5 9 6 8 1

```

Date de ieșire

```

1 Elicopterul 4 este pozitionat necorespunzator!
2 Elicopterul 5 este pozitionat necorespunzator!
3 2
4 2
5 1 3

```

Explicație 1

Dimensiunile citite sunt: $n = 7$, $m = 9$. Matricea citită de pe următoarele 7 linii este următoarea. În aceasta găsim $k = 6$ elicoptere, dintre care doar 4 valide.

```

1 1 1 1 1 1 1 1 1
2 0 0 0 0 1 1 1 0
3 0 0 1 0 1 1 1 0 0
4 1 1 1 0 1 1 0 1 1
5 0 0 1 1 1 1 0 1 1
6 1 1 1 1 1 1 0 1 1
7 1 1 1 1 1 1 0 0 1

```

Ipotenuzele triunghiurilor invalide au fost marcate mai jos cu culoarea roșu. Elicopterele valide sunt prezentate în matrice cu culoare galben, astfel:

- **Elicopterul 1:** (1,1), (3,3) și -1, poziționat în stânga sus
- **Elicopterul 2:** (1,9), (5,5) și 1, poziționat în dreapta sus
- **Elicopterul 3:** (5,1), (6,2) și 1, poziționat în stânga jos
- **Elicopterul 6:** (5,9), (6,8) și 1, poziționat în dreapta jos

1	1	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1	0
0	0	1	0	1	1	1	0	0
1	1	1	0	1	1	0	1	1
0	0	1	1	1	1	0	1	1
1	1	1	1	1	1	0	1	1
1	1	1	1	1	1	0	0	1

În continuare, nu vom lua în considerare elicopterele invalide. Astfel:

- Elicopterul 1 acoperă 4 zone afectate de umbră și 2 zone neafectate
- Elicopterul 2 acoperă în totalitate zone neafectate de umbră
- Elicopterul 3 acoperă 2 zone afectate de umbră și o zonă neafectată
- Elicopterul 6 acoperă în totalitate zone neafectate de umbră

Elicopterele 2 și 6 nu afectează niciun pătrățel de gazon.

Elicopterele 1 și 3 afectează fiecare mai mult de jumătate din numărul pătrățelilor asociate triunghiurilor dreptunghice și deci aterizează greșit. Elicopterul 1 face umbră la 6 pătrățele, dintre care afectate sunt 4. Elicopterul 3 face umbră la 3 pătrățele, dintre care afectate sunt două.

Exemplul 2

Date de intrare

```

1 8 8
2 1 0 0 1 1 0 1 0
3 0 0 1 1 0 1 0 1
4 0 0 1 1 1 0 1 0
5 1 0 0 0 1 1 1 1
6 1 0 0 0 1 1 0 1
7 1 0 0 0 0 1 1 0
8 1 1 1 1 0 1 1 1
9 0 0 1 1 1 0 0 0
10 5
11 3 2 6 5 -1
12 1 1 1 3 1
13 1 8 4 5 1
14 8 6 6 8 -1
15 3 1 7 1 1

```

Date de ieșire

```

1 Elicopterul 2 este pozitionat necorespunzator!
2 Elicopterul 5 este pozitionat necorespunzator!
3 0
4 2
5 1 4

```

Explicație 2

Dimensiunile citite sunt: $n = 8$, $m = 8$. Matricea citită de pe următoarele 8 linii este următoarea. În aceasta găsim $k = 5$ elicoptere, dintre care doar 3 valide.

```

1 1 0 0 1 1 0 1 0
2 0 0 1 1 0 1 0 1
3 0 0 1 1 1 0 1 0
4 1 0 0 0 1 1 1 1
5 1 0 0 0 1 1 0 1
6 1 0 0 0 0 1 1 0
7 1 1 1 1 0 1 1 1
8 0 0 1 1 1 0 0 0

```

Ipotenuzele triunghiurilor invalide au fost marcate mai jos cu culoarea roșu. Elicopterele valide sunt prezentate în matrice cu culoare galben, astfel:

- **Elicopterul 1:** (3,2), (6,5) și -1, poziționat în stânga
- **Elicopterul 3:** (1,8), (4,5) și 1, poziționat în dreapta sus
- **Elicopterul 4:** (8,6), (6,8) și -1, poziționat în dreapta jos

1	0	0	1	1	0	1	0
0	0	1	1	0	1	0	1
0	0	1	1	1	0	1	0
1	0	0	0	1	1	1	1
1	0	0	0	1	1	0	1
1	0	0	0	0	1	1	0
1	1	1	1	0	1	1	1
0	0	1	1	1	0	0	0

În continuare, nu vom lua în considerare elicopterele invalide. Astfel:

- Elicopterul 1 acoperă în totalitate zone cu umbră
- Elicopterul 3 acoperă 5 zone afectate de umbră și 5 zone neafectate
- Elicopterul 4 acoperă 4 zone afectate de umbră și 2 zone neafectate

Nu există niciun elicopter care nu afectează niciun pătrățel de gazon.

Elicopterele 1 și 4 afectează fiecare mai mult de jumătate din numărul pătrățelilor asociate triunghiurilor dreptunghice și deci aterizează greșit. Elicopterul 1 face umbră la 10 pătrățele, dintre care afectate sunt 10. Elicopterul 4 face umbră la 6 pătrățele, dintre care afectate sunt 4.

Elicopterul 3 face umbră la 10 pătrățele, dintre care afectate sunt 5, deci nu mai mult de jumătate din numărul lor total.

Regulament

Regulamentul general al temelor se găsește pe (OCW / Teme de casă). Vă rugăm să îl citiți integral înainte de continua cu regulile specifice acestei teme.

Arhivă

Soluția temei se va trimite ca o arhivă **zip**. Numele arhivei trebuie să fie de forma **Grupă_NumePrenume_TemaX.zip** - exemplu: 311CA_SavaVictor_Tema0.zip.

Arhiva trebuie să conțină în directorul **RĂDĂCINĂ** ****doar**** următoarele:

- Codul sursă al programului vostru (fișierele **.c** și eventual **.h**).
- Un fișier **Makefile** care să conțină regulile **build** și **clean**.
 - Regula **build** va compila codul vostru și va genera ****doar**** următoarele executabile:
 - * **window_join** pentru problema 1
 - * **akari** pentru problema 2
 - * **helicopters** pentru problema 3
 - Regula **clean** va șterge **toate** fișierele generate la build (executabile, binare intermediare etc).
- Un fișier **README** care să conțină prezentarea implementării alese de voi. **NU** copiați bucăți din enunț.

Arhiva temei **NU** va conține: fișiere binare, fișiere de intrare / ieșire folosite de checker, checker-ul, orice alt fișier care nu este cerut mai sus.

Numele și extensiile fișierelor trimise **NU** trebuie să conțină spații sau majuscule, cu excepția fișierului **README** (care are numele scris cu majuscule și nu are extensie).

Nerespectarea oricărei reguli din secțiunea **arhivă** aduce un punctaj **NUL** (0) pe temă.

Checker

Pentru corectarea acestei teme vom folosi scriptul **check** din arhiva **check_cookies_de_noel.zip** din secțiunea de resurse asociată temei. Vă rugăm să citiți **README-checker.md** pentru a ști cum să instalați și utilizați checker-ul.

Punctaj

Distribuirea punctajului pentru această temă este următoarea:

- Teste automate - Problema 1: 25p
- Teste automate - Problema 2: 25p
- Teste automate - Problema 3: 30p
- Claritatea și calitatea codului (inclusiv modularizare): 10p
- Claritatea explicațiilor din README: 10p

ATENȚIE! Punctajul maxim pe temă este **100p**. Acesta reprezintă 0.25p din nota finală la această materie. La această temă nu se vor acorda punctaje bonus.

Reguli și precizări

- Punctajul pe teste este cel acordat de script-ul **check**, rulat pe **Moodle**. Echipa de corectare își rezervă dreptul de a depuncta pentru orice încercare de a trece testele fraudulos (de exemplu prin hardcodare, etc).

- Punctajul pe calitatea explicațiilor și a codului se acordă în mai multe etape:
 - **corectare automată**
 - * Checker-ul va încerca să detecteze în mod automat probleme legate de coding style și alte aspecte de organizare a codului.
 - * Acesta va puncta cu maxim 20p dacă nu sunt probleme detectate în mod automat.
 - * Punctajul se va acorda proporțional cu numărul de puncte acumulate pe teste din cele 80p.
 - * Checker-ul poate să aplice însă și penalizări (exemplu pentru warninguri la compilare) sau alte probleme descoperite la runtime.
 - **corectare manuală**
 - * Tema va fi corectată manual și se vor verifica și alte aspecte pe care checker-ul nu le poate prinde. Recomandăm să parcurgeți cu atenție tutorialul de **coding-style** de pe ocw.cs.pub.ro.
 - * Codul sursă trebuie să fie însoțit de un fișier README care trebuie să conțină informațiile utile pentru înțelegerea funcționalității, modului de implementare și utilizare a programului. Acesta evaluează, de asemenea, abilitatea voastră de a documenta complet și concis programele pe care le produceți și va fi evaluat, în mod analog coding-style-ului, de către echipa de asistenți. În funcție de calitatea documentației, se vor aplica depunctări sau bonusuri.
 - * La corectarea manuală se va acorda un bonus de maximum 10 puncte pentru modularizare. Deprinderea de a scrie cod sursă de calitate, este un obiectiv important al materiei. Sursele greu de înțeles, modularizate neadecvat sau care prezintă hardcodări care pot afecta semnificativ mentenabilitatea programului cerut, pot fi depunctate adițional. Penalizarea pentru modularizare defectuoasă sau absentă complet, poate să fie oricât de mare.
 - * În această etapă se pot aplica depunctări oricât de mari (chiar și totale). Orice rezultat / punctaj acordat automat de checker, poate fi anulat sau suprascris de către echipa de PCLP la corectarea manuală.
 - * O temă care **NU** compilează cu -Wall -Wextra este depunctată la corectarea manuală cu 5p (punctajul echivalent pentru warnings). Echipa de PCLP vă recomandă să compilați cu -Wall -Wextra -Werror.
 - * **Tema trebuie să reprezinte exclusiv munca studentului!** Echipa va aplica regulamentele în vigoare pentru situațiile de fraudă / plagiat, pentru orice nerespectare a acestor reguli, incluzând, dar nelimitându-se la: copierea de la colegi (se aplică pentru ambele persoane implicate) sau din alte surse a unor părți din temă, prezentarea ca rezultat personal a oricărei bucăți de cod provenită din alte surse necitate sau neaprobate în prealabil (site-uri web, alte persoane, cod generat folosit AI / LLM-uri etc.).

Alte precizări

- Implementarea se va face în limbajul **C**, iar tema va fi compilată și testată **DOAR** într-un mediu **LINUX**. Nerespectarea acestor reguli aduce un punctaj **NUL**.
- Tema trebuie trimisă sub forma unei arhive pe site-ul cursului **curs.upb.ro**.
- Tema poate fi trimisă de oricâte ori fără depunctări până la deadline. Mai multe detalii se găsesc în regulamentul de pe [ocw](http://ocw.cs.pub.ro).
- O temă care **NU** compilează **NU** va fi punctată.
- O temă care compilează, dar care **NU** trece niciun test **NU** va fi punctată.
- Punctajul pe teste este cel acordat de **check** rulat pe **platforma remote a echipei de PCLP**. Echipa de corectare își rezervă dreptul de a depuncta pentru orice încercare de a trece testele fraudulos (de exemplu prin hardcodare).
- Ultima temă submisă pe Moodle poate fi rulată de către responsabili / echipa de PCLP de mai multe ori în vederea verificării faptului că nu aveți bug-uri în sursă. În cazul obținerii punctajelor diferite la rulări multiple, vom păstra minimul dintre punctaje. Vă recomandăm să verificați local tema de mai multe ori pentru a verifica că punctajul este mereu același, apoi să încărcați tema. De asemenea, verificați că punctajul de pe platformă este cel așteptat - singurul punctaj / feedback relevant este cel de pe platforma remote PCLP (**vmchecker-next**).